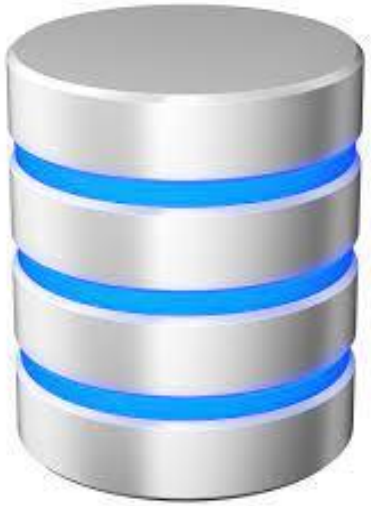


Lecture:

Functional Dependencies and Normalization for Relational Databases



Learning Objectives

- We first discuss informal guidelines for good relational design
- Then we discuss formal concepts of functional dependencies
- discuss normal forms
  - 1NF (First Normal Form)
  - 2NF (Second Normal Form)
  - 3NF (Third Normal Form)
  -

Informal Design Guidelines for Relational Databases

What is relational database design?

- The grouping of attributes to form "good" relation schemas

Two levels of relation schemas

- The logical "user view" level
- The storage "base relation" level

Design is concerned mainly with base relations

What are the criteria for "good" base relations?

Informal Design Guidelines for Relational Databases

Four *informal guidelines* that may be used as *measures to determine the quality* of relation schema design:

- Making sure that the semantics of the attributes is clear in the schema
- Reducing the redundant information in tuples
- Reducing the NULL values in tuples
- Disallowing the possibility of generating spurious tuples

GUIDELINE 1: Semantics of the Relation Attributes

GUIDELINE 1: Informally, each tuple in a relation should represent one entity or relationship instance. (Applies to individual relations and their attributes).

- Attributes of different entities (EMPLOYEEs, DEPARTMENTs, PROJECTs) should not be mixed in the same relation
- Only foreign keys should be used to refer to other entities
- Entity and relationship attributes should be kept apart as much as possible.

Bottom Line: *Design a schema that can be explained easily relation by relation. The semantics of attributes should be easy to interpret.*

Examples of Violating Guideline 1.

Redundant Information in Tuples and Update Anomalies

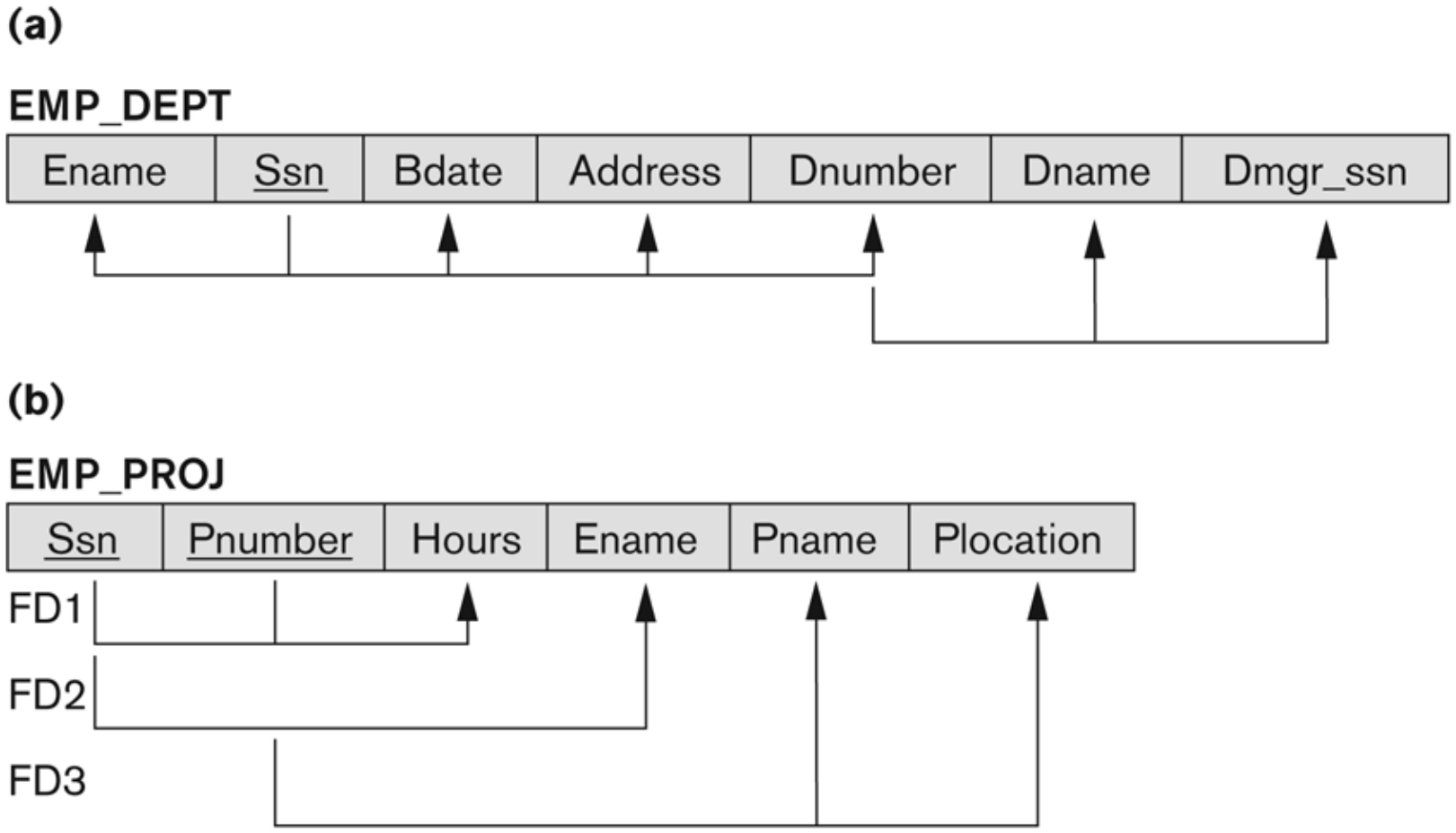
- Information is stored redundantly
  - Wastes storage
  - Causes problems with update anomalies
    - Insertion anomalies
    - Deletion anomalies
    - Modification anomalies

Anomalies

- A database anomaly is a fault in a database that usually emerges as a result of shoddy planning and storing everything in a flat database.
- The term anomaly is used to describe a discrepancy between two parts of a database
- A database anomaly is an inconsistency in the data resulting from an operation like an update, insertion, or deletion
- There can be inconsistencies when a record is held in multiple places and not all of the copies are updated.
- Anomalies are usually removed by normalizing the tables by splitting or joining them

Base Relations EMP_DEPT and EMP_PROJ

Two relation schemas suffering from update anomalies



Base Relations EMP_DEPT and EMP_PROJ formed after a Natural Join : with redundant information

Redundancy

Redundancy

EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston
888665555	20	Null	Borg, James E.	Reorganization	Houston

Redundancy

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

EXAMPLE OF AN UPDATE ANOMALY

Consider the relation:

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn

Update Anomaly:

In EMP_DEPT, if we change the value of one of the attributes of a particular department—say, the manager of department 5 -we must update the tuples of *all* employees who work in that department; otherwise, the database will become inconsistent

EXAMPLE OF AN INSERT ANOMALY

Consider the relation:

EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation

Insert Anomaly:

- Cannot insert a project unless an employee is assigned to it.

Conversely

- Cannot insert an employee unless a he/she is assigned to a project.

EXAMPLE OF AN DELETE ANOMALY

Consider the relation:

EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation

Delete Anomaly:

- When a project is deleted, it will result in deleting all the employees who work on that project.
- Alternately, if an employee is the sole employee on a project, deleting that employee would result in deleting the corresponding project.

repeat the department information
(“building” and “budget”)

ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

Fig. 4.1 An Instance of the relation Inst_dept.

Guideline to Redundant Information in Tuples and Update Anomalies

GUIDELINE 2:

- Design a schema that does not suffer from the insertion, deletion and update anomalies.

- If there are any anomalies present, then note them so that applications can be made to take them into account.

Null Values in Tuples

GUIDELINE 3:

- Relations should be designed such that their tuples will have as few NULL values as possible
- Attributes that are NULL frequently could be placed in separate relations (with the primary key)

Reasons for nulls:

- Attribute not applicable or invalid
- Attribute value unknown (may exist)
- Value known to exist, but unavailable

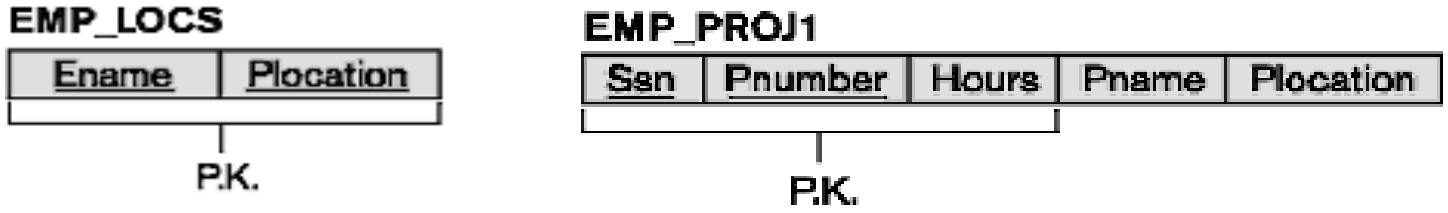
Spurious Tuples

- Bad designs for a relational database may result in erroneous results for certain JOIN operations
- Spurious Tuples are those rows in a table, which occur as a result of joining two tables in wrong manner
- They are extra tuples (rows) which might not be required
- They represent spurious information that is not valid.

  – No spurious tuples (meaningless or false) should be generated by doing a natural-join of any relations.

Spurious Tuples

(a)



(b)

EMP_LOCS

Ename	Plocation
Smith, John B.	Bellaire
Smith, John B.	Surgarland
Narayan, Ramesh K.	Houston
English, Joyce A.	Bellaire
English, Joyce A.	Surgarland
Wong, Franklin T.	Surgarland
Wong, Franklin T.	Houston
Wong, Franklin T.	Stafford
Zelaya, Alicia J.	Stafford
Jabbar, Ahmad V.	Stafford
Wallace, Jennifer S.	Stafford
Wallace, Jennifer S.	Houston
Borg, James E.	Houston

EMP_PROJ1

Ssn	Pnumber	Hours	Pname	Plocation
123456789	1	32.5	ProductX	Bellaire
123456789	2	7.5	ProductY	Sugarland
666884444	3	40.0	ProductZ	Houston
453453453	1	20.0	ProductX	Bellaire
453453453	2	20.0	ProductY	Sugarland
333445555	2	10.0	ProductY	Sugarland
333445555	3	10.0	ProductZ	Houston
333445555	10	10.0	Computerization	Stafford
333445555	20	10.0	Reorganization	Houston
999887777	30	30.0	Newbenefits	Stafford
999887777	10	10.0	Computerization	Stafford
987987987	10	35.0	Computerization	Stafford
987987987	30	5.0	Newbenefits	Stafford
987654321	30	20.0	Newbenefits	Stafford
987654321	20	15.0	Reorganization	Houston
888665555	20	NULL	Reorganization	Houston

Example of Spurious Tuples contd

	Ssn	Pnumber	Hours	Pname	Plocation	Ename
	123456789	1	32.5	ProductX	Bellaire	Smith, John B.
*	123456789	1	32.5	ProductX	Bellaire	English, Joyce A.
	123456789	2	7.5	ProductY	Sugarland	Smith, John B.
*	123456789	2	7.5	ProductY	Sugarland	English, Joyce A.
*	123456789	2	7.5	ProductY	Sugarland	Wong, Franklin T.
	666884444	3	40.0	ProductZ	Houston	Narayan, Ramesh K.
*	666884444	3	40.0	ProductZ	Houston	Wong, Franklin T.
*	453453453	1	20.0	ProductX	Bellaire	Smith, John B.
	453453453	1	20.0	ProductX	Bellaire	English, Joyce A.
*	453453453	2	20.0	ProductY	Sugarland	Smith, John B.
	453453453	2	20.0	ProductY	Sugarland	English, Joyce A.
*	453453453	2	20.0	ProductY	Sugarland	Wong, Franklin T.
*	333445555	2	10.0	ProductY	Sugarland	Smith, John B.
*	333445555	2	10.0	ProductY	Sugarland	English, Joyce A.
	333445555	2	10.0	ProductY	Sugarland	Wong, Franklin T.
*	333445555	3	10.0	ProductZ	Houston	Narayan, Ramesh K.
	333445555	3	10.0	ProductZ	Houston	Wong, Franklin T.
	333445555	10	10.0	Computerization	Stafford	Wong, Franklin T.
*	333445555	20	10.0	Reorganization	Houston	Narayan, Ramesh K.
	333445555	20	10.0	Reorganization	Houston	Wong, Franklin T.

*
*
*

Figure 10.6
Result of applying NATURAL JOIN to the tuples above the dotted lines in EMP_PROJ1 and EMP_LOCS of Figure 10.5. Generated spurious tuples are marked by asterisks.

Spurious Tuples

	Ssn	Pnumber	Hours	Pname	Plocation	Ename
	123456789	1	32.5	ProductX	Bellaire	Smith, John B.
*	123456789	1	32.5	ProductX	Bellaire	English, Joyce A.
	123456789	2	7.5	ProductY	Sugarland	Smith, John B.
*	123456789	2	7.5	ProductY	Sugarland	English, Joyce A.
*	123456789	2	7.5	ProductY	Sugarland	Wong, Franklin T.
	666884444	3	40.0	ProductZ	Houston	Narayan, Ramesh K.
*	666884444	3	40.0	ProductZ	Houston	Wong, Franklin T.
*	453453453	1	20.0	ProductX	Bellaire	Smith, John B.
	453453453	1	20.0	ProductX	Bellaire	English, Joyce A.
*	453453453	2	20.0	ProductY	Sugarland	Smith, John B.
	453453453	2	20.0	ProductY	Sugarland	English, Joyce A.
*	453453453	2	20.0	ProductY	Sugarland	Wong, Franklin T.
*	333445555	2	10.0	ProductY	Sugarland	Smith, John B.
*	333445555	2	10.0	ProductY	Sugarland	English, Joyce A.
	333445555	2	10.0	ProductY	Sugarland	Wong, Franklin T.
*	333445555	3	10.0	ProductZ	Houston	Narayan, Ramesh K.
	333445555	3	10.0	ProductZ	Houston	Wong, Franklin T.
	333445555	10	10.0	Computerization	Stafford	Wong, Franklin T.
*	333445555	20	10.0	Reorganization	Houston	Narayan, Ramesh K.
	333445555	20	10.0	Reorganization	Houston	Wong, Franklin T.

*
*
*

Figure 10.6
Result of applying NATURAL JOIN to the tuples above the dotted lines in EMP_PROJ1 and EMP_LOCS of Figure 10.5. Generated spurious tuples are marked by asterisks.

Spurious Tuples (cont.)

GUIDELINE 4:

- Design relation schemas so that they can be joined with equality conditions on attributes that are appropriately related (primary key, foreign key) pairs in a way that guarantees that no spurious tuples are generated.

- Avoid relations that contain matching attributes that are not (foreign key, primary key) combinations because joining on such attributes may produce spurious tuples

Functional Dependencies (FDs)

- FDs are used to specify *formal measures* of the "goodness" of relational designs
- FDs and keys are used to define **normal forms** for relations
- FDs are **constraints** that are derived from the *meaning* and *interrelationships* of the data attributes

- A set of attributes X *functionally determines* a set of attributes Y if the value of X determines a unique value for Y

Functional Dependencies (cont.)

- **Definition.**

A **functional dependency (FD)**, denoted by $X \rightarrow Y$, between two sets of attributes X and Y that are subsets of R specifies a *constraint* on the possible tuples that can form a relation state r of R. The constraint is that, for any two tuples $t1$ and $t2$ in r that have $t1[X] = t2[X]$, they must also have $t1[Y] = t2[Y]$.

the values of the X component of a tuple uniquely (or **functionally**) *determine* the values of the Y component

Functional Dependencies (cont.)

A functional dependency (FD) on a relation R is a statement of the form “If two tuples of R agree on all of th e attributes $A_1A_2…A_n$ (i.e., the tuples have the same values in their respective components for each f these attributes), then they must also agree on all the attributes $B_1B_2…B_m$. We write this FD formally as

$A_1A_2…A_n \rightarrow B_1B_2…B_m$ and say that “$A_1A_2…A_n$ functionally determine $B_1B_2…B_m$” It is common for the right si de of an FD to be a single attribute.

Functional Dependencies (cont.)

consider the instance of the relation.

Moviesl (title, year, length, genre, studioName, starName)

Which of the following is a true FD.

Title, year → length, genre, studioName (FD)

title, year →□starName (NOT)

title	*year*	*length*	*genre*	*studioName*	*starName*
Star Wars	1977	124	SciFi	Fox	Carrie Fisher
Star Wars	1977	124	SciFi	Fox	Mark Hamill
Star Wars	1977	124	SciFi	Fox	Harrison Ford
Gone With the Wind	1939	231	drama	MGM	Vivien Leigh
Wayne's World	1992	95	comedy	Paramount	Dana Carvey
Wayne's World	1992	95	comedy	Paramount	Mike Meyers

Fig.4.2 An Instance of Relation Movies 1

Examples of FD constraints (cont.)

roll_no	name	dept_name	dept_building
42	abc	CO	A4
43	pqr	IT	A3
44	xyz	CO	A4
45	xyz	IT	A3
46	mno	EC	B2
47	jkl	ME	B2

valid functional dependencies

- roll_no → { name, dept_name, dept_building }
- dept_name → dept_building
- {roll_no, name} ⇢ {dept_name, dept_building}

Invalid functional dependencies

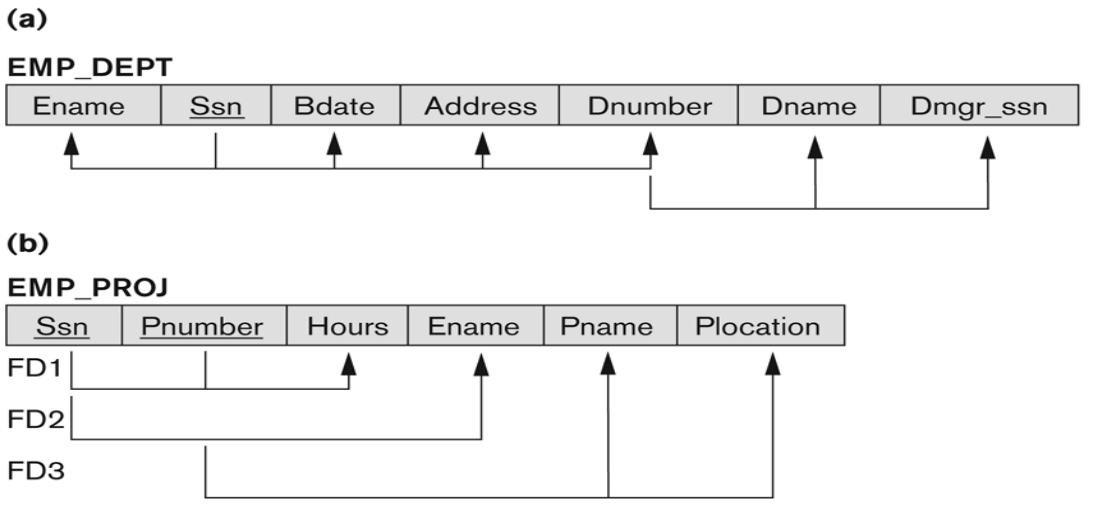
- name → dept_name
- dept_building → dept_name
- dept_building → roll_no

Examples of FD constraints (cont.)

- Social security number determines employee name
  - SSN -> ENAME

- Project number determines project name and location
  - PNUMBER -> {PNAME, PLOCATION}

- Employee ssn and project number determines the hours per week that the employee works on the project
  - {SSN, PNUMBER} -> HOURS



Trivial Functional Dependencies

Definition 4.1. (Trivial Functional Dependencies): A FD is called trivial if its right side is a subset of its left side.

That is, They are the FD's $A_1A_2\ldots A_n \rightarrow B_1B_2\ldots B_m$ such that $\{B_1B_2\ldots B_m\} \subseteq \{A_1A_2\ldots A_n\}$.

For example, title, year $\rightarrow$ title is a trivial FD, as is

title $\rightarrow$title.

- two tuples that agree in all of $A_1A_2\ldots A_n$ agree in a subset of them."

Reasoning About Functional Dependency:

- Given a set of FDs F, we can **infer** additional FDs that hold whenever the FDs in F hold
  - Rule1. (**Transitive**) If X -> Y and Y -> Z, then X -> Z
    if roll_no → dept_name & dept_name → dept_building,
    then roll_no → dept_building is also valid.
  - Rule2. **(Decomposition)** If X -> YZ, then X -> Y and X -> Z
  - Rule3(**Union):** If X -> Y and X -> Z, then X -> YZ

the set of FD's:

title, year → length
title, year →genre
title, year → studioName

is equivalent to the single

FD: title,year →length, genre, studioName

(Notation: XZ stands for X U Z)

Inference Rules for FDs (cont.)

- RULE 4: If FD's X ->B1 & Y ->B2, then they can be replaced by FD XY ->B1B2.

- Rule5. (**Augmentation**) If X -> Y, then XZ -> YZ
  - If {roll_no, name} → dept_building is valid,

    hence {roll_no, name, dept_name} → {dept_building, dept_name} is als o valid

Equivalence of Sets of FDs

- Two sets of FDs F and G are **equivalent** if
  - Every FD in F can be inferred from G, an d
  - Every FD in G can be inferred from F

example:

S = {A → B, A → C}

T = {A → BC}

S and T are **equivalent ??**

- From S, derive T

From A → B and A → C

Combine them: A → BC

From T, derive S

From A → BC

→ We split into:

A → B

A → C

Example

Example 4.1: Given: F = {A →BC, CD→ F, BE →D} for R(A,B,C,D,E,F). Is AE →CDF true?

BE →D (given)
BCE→ CD (accumulation of BE -> D)
BCE →D (decomposition)
A →BC (given)
AE→ BCE (accumulation)
AE→ D (transitivity)
AE → BCDE (combining rule)
AE →CD (decomposition)
AE →F (transitivity)
AE →CDF (combining rule)

Computing the Closure of Attributes

- **Let A = {A1, A2, ..., An} be a set of attributes in a relation R and X be a set of functional dependency over attributes of R. The closure of A under the FD's in X, denoted A+, contains, in addition to A, new attributes determined by A1, A2, ..., An and generated using FDs in X.**
- **The closure of an attribute (X^+) is the complete set of attributes that you can derive from X using the given functional dependencies.**

Algorithm 4.1: Closure of a Set of Attributes.
INPUT: A set of attributes A={A1, A2, ..., An} and a set of FD's X.
OUTPUT: The closure A+ = {A1, A2, ..., An}+.
If necessary, split the FD's of X, so each FD in X has a single attribute on the right.

1. Let A+ be a set of attributes that eventually will become the closure. Initialize A+ to be {A1, A2, ..., An}.
2. Repeatedly search for some FD B1B2...Bm · C such that all of B1, B2, ... ,Bm are in the set of attributes A+, but C is not. Add C to the set A+ and repeat the search. Since A+ can only grow, and the number of attributes of any relation schema must be finite, eventually nothing more can be added to A+, and this step ends.
3. The set A+, after no more attributes can be added to it, is the
4. correct value

Computing the Closure of Attributes(cont.)

example1: Let us consider a relation R with attributes A, B, C, D, E, and F. Suppose that this relation has the FD's

AB → C,

BC → AD,

D → E,

CF → B.

compute $\{A, B\}^+$

1. split BC → AD into BC → A and BC → D according to RULE 2
2. start with {A, B}+ = {A, B}.
3. both attributes on the left side of FD AB → C are in {A, B}+ so {A, B}+ = {A, B,C}
4. the left sides of BC →A and BC →D are now contained in {A, B}+ so add A and D to {A, B}+.
5. now {A, B}+ = {A, B,C,D}
6. the left sides of D →E are now contained in {A, B}+ So add E to {A, B}+, which is now {A, B}+= {A, B, C , D, E}
7. the FD CF →B can not be used, because its left side never becomes contained in {A, B}+.
8. the final result is $\{A, B\}^+ = \{A, B, C, D, E\}$

Computing the Closure of Attributes(cont.)

EX2: F = {A → B, B → D, AB → C}
Compute: $(A)^+$

Start: $A^+ = \{ A \}$
1. A → B ⇒ add B then $A^+ = \{ A, B \}$
2. B → D ⇒ add D then $A^+ = \{ A, B, D \}$
3. AB → C?

We have A and B ⇒ add C then $A^+ = \{ A, B, C, D \}$

Final answer: $A^+ = \{ A, B, C, D \}$

EX3: Consider the relation: **R(A, B, C, D)**
And the functional dependencies:

F = {A → B, BC → D, D → C}

compute the closure of A(**A^+**)

Start:

$A^+ = \{ A \}$

1. FD 1: A → B CAN be used

A is in A^+, so we can apply it.
Add B: $A^+ = \{ A, B \}$
2. FD 2: BC → D CANNOT be used yet

To use BC → D, we need both B and C.
We have B and We do not have C
So FD cannot be used at this stage.
It "contains" attributes not in the closure yet therefore it is unusable.
3. FD 3: D → C CANNOT be used yet
To use D → C we need D.
We do not have D so Cannot apply it.
4. Final Closure $A^+ = \{ A, B \}$

Computing the Closure of Attributes(cont.)

- By computing the closure of any set of attributes, we can test whether any given FD $A_1A_2...A_n \rightarrow B$ follows from a set of FD's X
- $A_1A_2...A_n \rightarrow B_1B_2...B_m$ follows from set of FD's X if and only if all of $B_1B_2...B_m$ are in $\{A_1A_2...A_n\}^+$.

example1: Let us consider a relation R with attributes A, B, C, D, E, and F.
Suppose that this relation has the FD's
AB → C,
BC → AD,
D → E,
CF → B.

1. AB →D follows from these FD's??
2. D →A follows from these FD's??

1. We compute $\{A, B\}^+=\{A, B, C, D, E\}$.
   D in $\{A, B\}^+$
   then AB →D follows from these FD's

2. We compute $\{D\}^+=\{D, E\}$.
   A not in $\{D\}^+$
   then D →A dosn't follows from these FD's

Another Definition of Keys of A relation

Definition 4.2 (Keys): We say a set of one or more attributes $\{A_1A_2\ldots A_n\}$ is a key for a relation R if:

1. Those attributes functionally determine all other attributes of the relation. That is, it is impossible for two distinct tuples of R to agree on all of $A_1A_2\ldots A_n$.
2. No proper subset of $\{A_1A_2\ldots A_n\}$ functionally determines all other attributes of R; i.e., a key must be minimal.

Another Definition of Keys of A relation

Moviesl (title, year, length, genre, studioName, starName)

title	*year*	*length*	*genre*	*studioName*	*starName*
Star Wars	1977	124	SciFi	Fox	Carrie Fisher
Star Wars	1977	124	SciFi	Fox	Mark Hamill
Star Wars	1977	124	SciFi	Fox	Harrison Ford
Gone With the Wind	1939	231	drama	MGM	Vivien Leigh
Wayne's World	1992	95	comedy	Paramount	Dana Carvey
Wayne's World	1992	95	comedy	Paramount	Mike Meyers

Fig.4.2 An Instance of Relation Movies 1

Show that attributes {title, year, starName} form a key for the relation Moviesl

prime & nonprime attributes

An attribute of relation schema R is called

- a prime attribute of R if it is a member of some candidate ke y of R.
- An attribute is called nonprime if it is not a prime attribute—that is, if it is not a member of any candidate key

Consider the relation:
R(A, B, C, D)with the following functional dependencies:

F = {A → B, C → D, AC → B} find the canditate key

Compute AC^+
AC^+={A,B,C,D}
AC is candidate

So the only candidate key is:
Candidate Key = AC

Check if any subset (A or C alone) is a key:
A^+ = { A, B } → not all attributes
C^+ = { C, D } → not all attributes

Normal Forms Based on Primary Keys

1. Normalization of Relations
2. Practical Use of Normal Forms
3. Definitions of Keys and Attributes Participating in Keys
4. First Normal Form
5. Second Normal Form
6. Third Normal Form

Normalization of Relations

Normalization:

- The process of decomposing unsatisfactory "bad" relations by breaking up their attributes into smaller relations
- reduces data redundancy and eliminates undesirable characteristics like Insertion, Update and Deletion Anomalies.

Normal form:

- Condition using keys and FDs of a relation to certify whether a relation schema is in a particular normal form

Normalization of Relations (cont.)

2NF, 3NF, BCNF

- based on keys and FDs of a relation schema

4NF

- based on keys, multi-valued dependencies : MVDs; 5NF based on keys, join dependencies : JDs

Practical Use of Normal Forms

- **Normalization**
  - is carried out in practice so that the resulting designs are of high quality and meet the desirable properties
  - The practical utility of these normal forms becomes questionable when the constraints on which they are based are hard to understand or to detect
  - The database designers need not normalize to the highest possible normal form
    - (usually up to 3NF, BCNF)

Definitions of Keys and Attributes Participating in Keys (cont.)

- If a relation schema has more than one key, each is called a **candidate** key.
  - One of the candidate keys is *arbitrarily* designated to be the **primary key**, and the others are called **secondary keys**.

- A **Prime attribute** must be a member of *some* candidate key
- A **Nonprime attribute** is not a prime attribute—that is, it is not a member of any candidate key.

First Normal Form

Disallows

- multivalued attributes
- Composite attribtues
- **nested relations**; attributes whose values for an *individual*

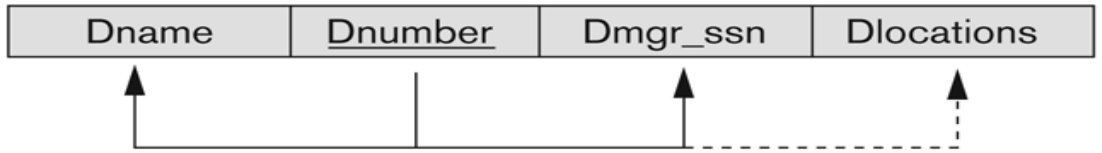
  tuple are non-atomic

Considered to be part of the definition of relation

Normalization into 1NF

(a)

DEPARTMENT



(b)

DEPARTMENT

Dnumber → Dlocations

Dname	Dnumber	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

Dlocations is not functionally dependent on the primary key *Dnumber*

(c)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

Fig 4.3 Normalization into 1NF.
(a) A relation schema that is not in **1NF**.
(b) Sample state of relation **DEPARTMENT.**
(c) **1NF** version of the same relation with redundancy

Normalization of nested relations into 1NF

(a)

EMP_PROJ

		Projs	
Ssn	Ename	Pnumber	Hours

(b)

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, AliciaJ.	30	30.0
		10	10.0
98798798 7	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)

EMP_PROJ1

Ssn	Ename

EMP_PROJ2

Ssn	Pnumber	Hours

Fig. 4.4 Normalizing nested relations into 1NF.

(a) Schema of the **EMP_PROJ** relation with a nested relation attribute **PROJS**.

(b) Sample extension of the **EMP_PROJ** relation showing nested relations within each tuple.

(c) Decomposition of **EMP_PROJ** into relations **EMP_PROJ1** and **EMP_PROJ2** by propagating the primary key.

Second Normal Form

Uses the concepts of Full **FDs, primary key**

Definitions

- **Prime attribute:** An attribute that is member of the primary key K
- **Full functional dependency:** a FD Y -> Z where removal of any attribute from

  Y means the FD does not hold any more

Examples:

- {SSN, PNUMBER} -> HOURS is a full FD since neither SSN -> HOURS nor PNUMBER -> HOURS hold
- {SSN, PNUMBER} -> ENAME is not a full FD (it is called a partial dependency)

  since SSN -> ENAME also holds

Second Normal Form (cont.)

- A relation schema R is in **second normal form (2NF)** if it is
  - in 1NF and
  - every non-prime attribute A in R is fully functionally dependent on the primary key
- in other words has no partial dependency. That is, all non-key attributes are fully dependent on a primary key.
- R can be decomposed into 2NF relations via the process of
- 2NF normalization

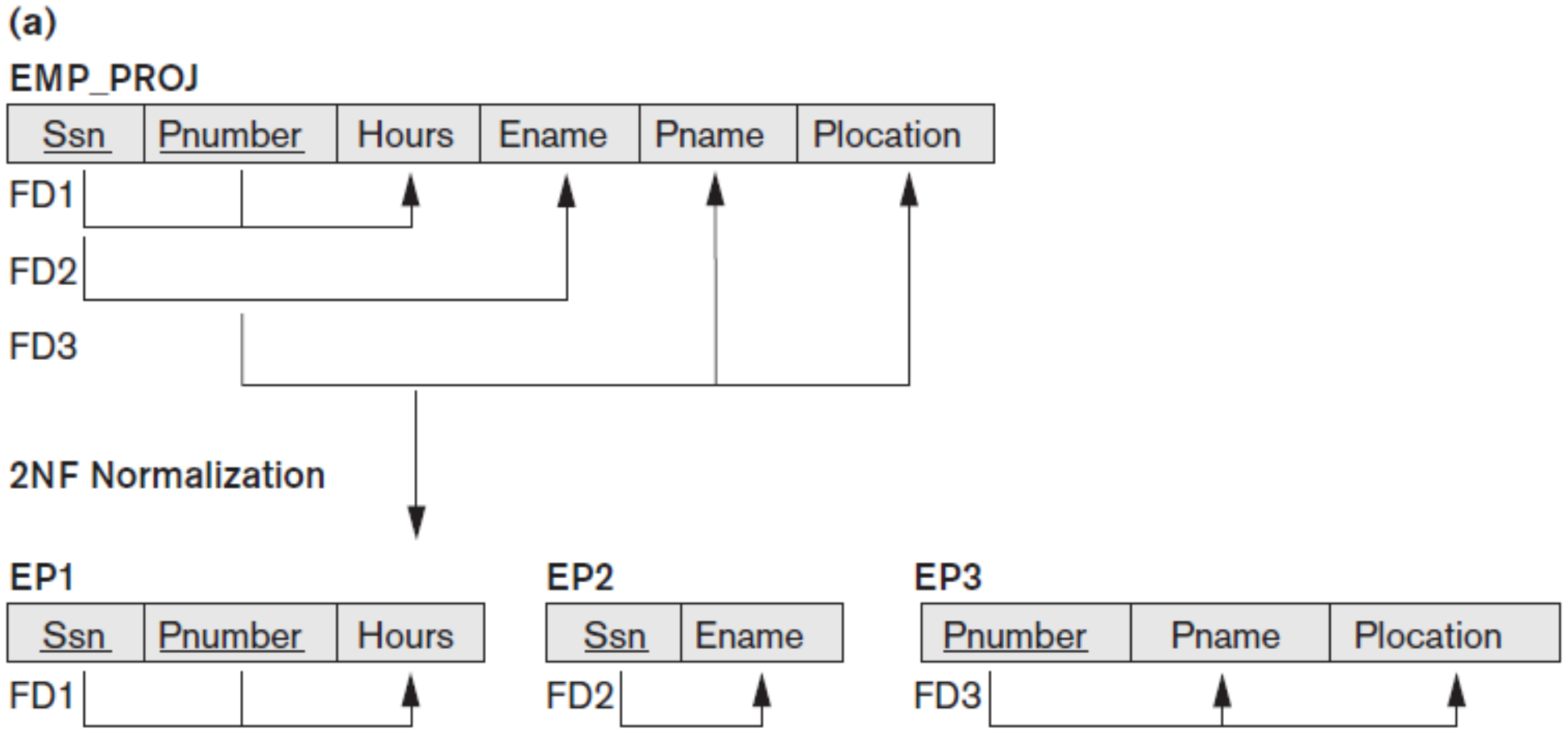
{*Ssn, Pnumber*} → *Hours* *full* functional dependency

(neither *Ssn* → *Hours* nor *Pnumber* → *Hours* holds)

{*Ssn, Pnumber*} → *Ename* partial dependency

Ssn → *Ename* holds.

Normalizing into 2NF



Third Normal Form

Definition:

- **Transitive functional dependency:** a FD X -> Z that c an be

  derived from two FDs X -> Y and Y -> Z

Examples:

- SSN -> DMGRSSN is a **transitive** FD
  - Since SSN -> DNUMBER and DNUMBER -> DMGRSSN hold
- SSN -> ENAME is **non-transitive**
  - Since there is no set of attributes X where SSN -> X and X -> ENAME

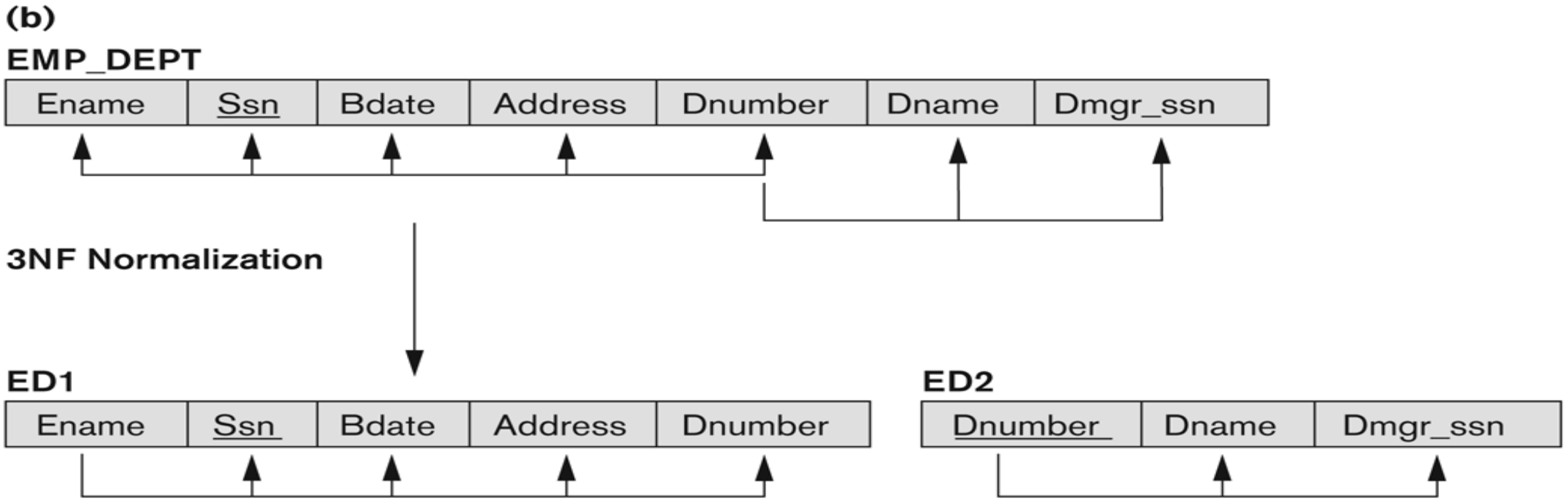
Third Normal Form (cont.)

- A relation schema R is in **third normal form (3NF)** if it is in 2NF *and* no non-prime attribute A in R is transitively dependent on the primary key
  R can be decomposed into 3NF relations via the process of 3NF normalization (remove these transitive dependencies.)

NOTE:

- In X -> Y and Y -> Z, with X as the primary key, we consider this a problem only if Y is not a candidate key.
- When Y is a candidate key, there is no problem with the transitive dependency .
- E.g., Consider EMP (SSN, Emp#, Salary).
  - Here, SSN -> Emp# -> Salary and Emp# is a candidate key.

Normalizing into 3NF



Ssn -> Dnumber and Dnumber ->Dmgr_ssn

Normal Forms Defined Informally

2nd normal form

- All attributes depend on **the whole key**

3rd normal form

- All attributes depend on **nothing but the key**

example

EMPLOYEE_ID	NAME	JOB_CODE	JOB	STATE_CODE	HOME_STATE
E001	Alice	J01	Chef	26	Michigan
E001	Alice	J02	Waiter	26	Michigan
E002	Bob	J02	Waiter	56	Wyoming
E002	Bob	J03	Bartender	56	Wyoming
E003	Alice	J01	Chef	56	Wyoming

2NF
??

name, home_state, and state_code are dependent on employee_id (a part of prim ary composite key). So, the table is not in 2NF

EMPLOYEE_ID	JOB_CODE
E001	J01
E001	J02
E002	J02
E002	J03
E003	J01

MPLOYEE_ID	NAME	STATE_CODE	HOME_STATE
E001	Alice	26	Michigan
E002	Bob	56	Wyoming
E003	Alice	56	Wyoming

JOB_CODE	JOB
J01	Chef
J02	Waiter
J03	Bartender

2NF
??

home_state is now dependent on state_code. So, if you know the state_code, then you can find the home_state value.

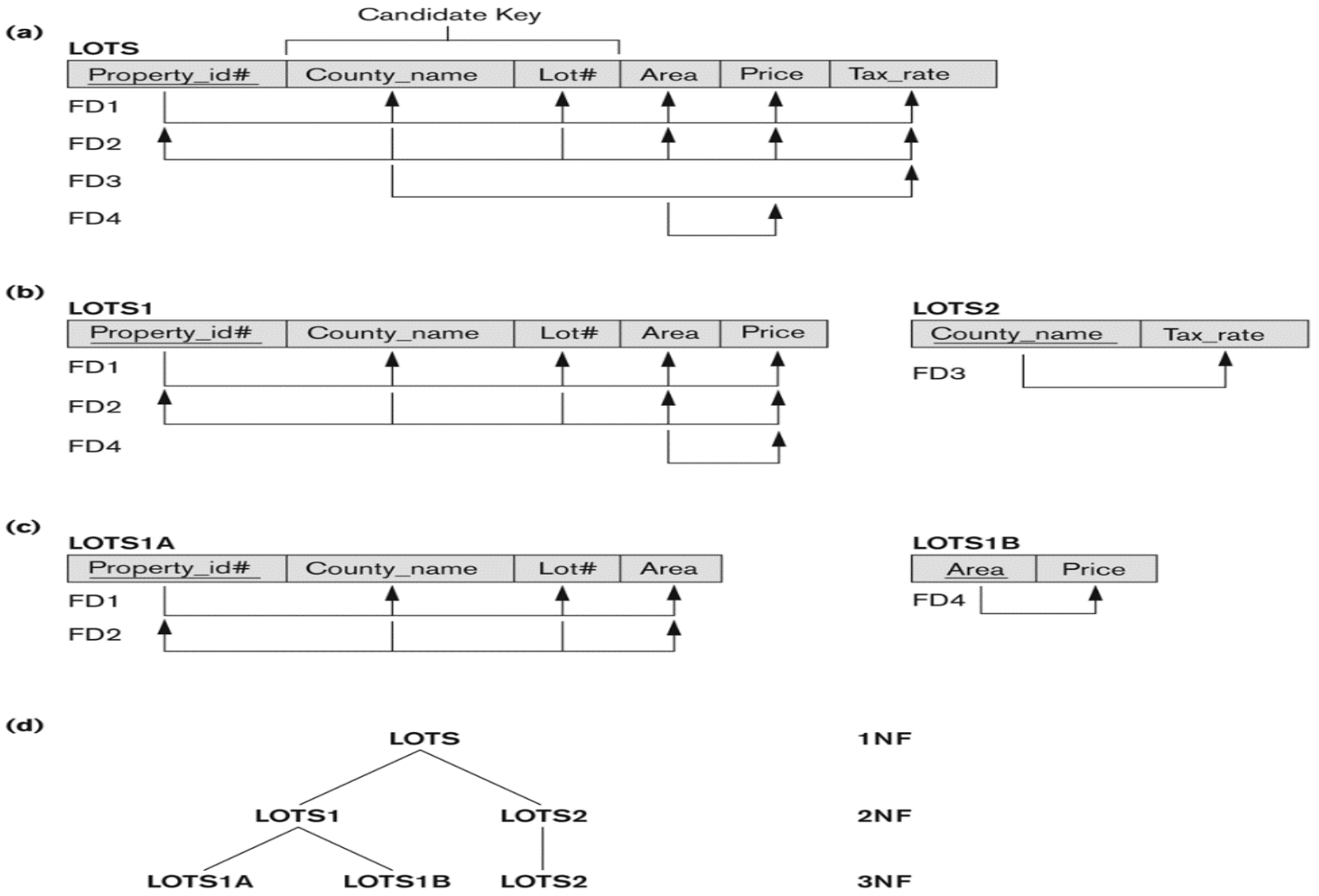
EMPLOYEE_ID	JOB_CODE
E001	J01
E001	J02
E002	J02
E002	J03
E003	J01

EMPLOYEE_ID	NAME	STATE_CODE
E001	Alice	26
E002	Bob	56
E003	Alice	56

JOB_CODE	JOB
J01	Chef
J02	Waiter
J03	Bartender

TATE_CODE	HOME_STATE
26	Michigan
56	Wyoming

Successive Normalization of LOTS into 2NF and 3NF



SUMMARY OF NORMAL FORMS based on Primary Keys

Summary of Normal Forms Based on Primary Keys and Corresponding Normalization

Normal Form	Test	Remedy (Normalization)
First (1NF)	Relation should have no multivalued attributes or nested relations.	Form new relations for each multi-valued attribute or nested relation.
Second (2NF)	For relations where primary key contains multiple attributes, no nonkey attribute should be functionally dependent on a part of the primary key.	Decompose and set up a new relation for each partial key with its dependent attribute(s). Make sure to keep a relation with the original primary key and any attributes that are fully functionally dependent on it.
Third (3NF)	Relation should not have a nonkey attribute functionally determined by another nonkey attribute (or by a set of nonkey attributes). That is, there should be no transitive dependency of a nonkey attribute on the primary key.	Decompose and set up a relation that includes the nonkey attribute(s) that functionally determine(s) other nonkey attribute(s).

Chapter Summary

- Informal Design Guidelines for Relational Databases
- Functional Dependencies (FDs)
  - Definition, Inference Rules, Equivalence of Sets of FDs, Minimal Sets of FDs
- Normal Forms Based on Primary Keys
- General Normal Form Definitions (For Multiple Keys)


THANK
YOU


Any
Question?